

ROTEIRO PARA AULAS PRÁTICAS DE SQL

Disciplina: **Banco de Dados**
Professor: **Howard Roatti**
Semestre: **2025/2**
Data da Criação: **20/09/2022**
Última Atualização: **10/09/2025**

Sumário

Objetivo	3
Convenções Adotadas e Pré-Instruções:.....	4
Orientações Iniciais	6
Preparação do ambiente	7
Parte 1 - DDL	10
1 - Create	10
1.1 Tables	10
1.2 View.....	14
1.3 Index.....	16
1.4 Sequences	17
2 - Alter Table	18
2.1 - Add Column.....	18
2.2 - Modify Column.....	19
2.3 - Rename Column	19
2.4 - Rename Table.....	20
2.5 - Drop Column	20
2.6 - Add Primary Key	21
2.7 - Add Constraint	22
3 - Drop.....	23
3.1 - Tables	23
3.2 - View.....	24
3.3 - Index.....	24
3.4 - Constraints	24
3.5 - Sequences	25
Seção Final da Parte 1	25
Parte 2 - DML.....	27
Seção 1 - Inserção.....	27

Seção 2 - Atualização.....	32
Seção 3 - Exclusão	34
Seção 4 - Consultas.....	35
Parte 3 - DML – <i>Queries (Select)</i>	37
Nível 1 - Cláusula Where	37
Nível 2 - Joins.....	38
Nível 3 - Funções Agregadas	39
Nível 4 - Subqueries.....	40
Nível 5 - Outer Joins	41
Nível 6 - Combinação dos Níveis Anteriores (Nível Hard)	41
Parte 4 – PL/SQL.....	43
Seção 1 - Function	43
Seção 2 - Procedure	45
Seção 3 - Triggers	48

Objetivo

Este roteiro tem como objetivo proporcionar ao aluno a vivência prática da criação e execução de instruções SQL, utilizando como base a IDE **DBeaver** conectada a um banco de dados relacional (preferencialmente Oracle, mas aplicável a outros SGBDs, como PostgreSQL e MySQL via Docker). Além disso, incentiva a experimentação da **automação de scripts SQL com Python**, integrando conhecimentos técnicos com boas práticas de desenvolvimento.

Ao final das atividades, espera-se que o aluno:

1. Compreenda a importância e a aplicabilidade prática das declarações SQL no desenvolvimento de sistemas;
2. Leia atentamente as instruções de cada seção, identificando dúvidas e buscando esclarecimentos;
3. Associe as sintaxes SQL aos exemplos fornecidos, interpretando e adaptando para diferentes contextos;
4. Desenvolva scripts SQL a partir das atividades propostas, organizando-os para reuso e versionamento;
5. Explore ao menos uma IDE para execução e depuração de comandos SQL, com destaque para o DBeaver;
6. Experimente a execução automatizada de scripts SQL com Python, integrando-os a projetos reais;
7. Generalize os conhecimentos adquiridos, aplicando SQL em contextos acadêmicos e profissionais;
8. Relacione o uso de scripts Python com outras linguagens de programação em cenários de integração com bancos de dados.

Convenções Adotadas e Pré-Instruções:

Para facilitar a leitura e execução dos comandos ao longo do roteiro, adota-se a seguinte convenção:

- Tudo o que estiver **entre colchetes []** é **opcional**;
- Tudo o que estiver **entre sinais de menor e maior < >** **deve ser substituído obrigatoriamente** por um valor adequado;
- Tudo o que estiver **entre {OPCAO1 | OPCA02 | ... }** **deve ser substituído obrigatoriamente** por uma das opções;
- As linguagens utilizadas neste roteiro são:
 - **SQL** (padrão ANSI, com foco em Oracle, mas compatível com PostgreSQL e MySQL);
 - **Python**, para automação da execução dos scripts SQL;
- Todas as instruções SQL **devem ser finalizadas com ponto e vírgula ;**

Execução de comandos SQL

- A execução pode ser feita diretamente pela **IDE DBeaver**, conectada ao banco de dados via JDBC.
- Para executar uma instrução individual no DBeaver, selecione o comando e pressione **Ctrl + Enter**.
- Para executar múltiplas instruções em sequência, selecione o bloco desejado e pressione **Alt + X**.

Scripts Python

- Os scripts em Python podem ser desenvolvidos e executados no **VS Code** ou outra IDE de sua preferência.
- A execução pode ser feita via terminal com:

python nome_do_script.py

ou

python3 nome_do_script.py

- Os scripts se baseiam em um módulo utilitário (OracleQueries, por exemplo), que já se encontra disponível no repositório oficial do professor. O módulo pode ser adaptado conforme o SGBD utilizado.

Boas práticas

- Salve todos os scripts desenvolvidos (SQL e Python) com nomes significativos e organizados em pastas temáticas.
- Sempre que possível, utilize **controle de versão com Git**. Sugere-se o uso do GitHub para armazenar seus scripts e acompanhar sua evolução.
- Caso queira começar, seguem algumas referências úteis:
 - [Como instalar o Git no Ubuntu](#)
 - [Como usar o Git de forma eficaz](#)
 - [Comandos Git básicos e avançados](#)
 - **Principais comandos para o controle de versionamento:** *git status* (verificar as alterações), *git add/delete* (adicionar ou remover as alterações), *git commit* -m "comentario", *git push* (leva as alterações para o repositório na origem), *git pull* (recupera as ultimas atualizações do repositório), *git merge* (mescla as branches), *git branch* (cria e acompanha branches), *git diff* (analisa diferenças de versões).

Orientações Iniciais

Este roteiro prático foi desenvolvido para acompanhar a sua jornada na criação e manipulação de objetos em bancos de dados relacionais utilizando SQL. As instruções abaixo visam garantir que você aproveite ao máximo as aulas, com um ambiente funcional e boas práticas de organização.

Etapas fundamentais de cada parte do roteiro

- **Parte 1 – DDL:** Você deverá criar as estruturas de tabelas, views, índices, constraints e sequences.
 - Ao final dessa parte, salve um **script SQL único** com todas as instruções criadas, para reutilização nas próximas etapas e testes com Python.
- **Parte 2 – DML:** A partir do script da Parte 1, você irá:
 - Inserir dados;
 - Atualizar registros;
 - Realizar exclusões e consultar resultados.
 - Ao final de cada seção (inserção, atualização, exclusão), gere **scripts separados** para reaproveitamento posterior, incluindo via Python.
- **Parte 3 – Consultas (SELECT):** Com base nas tabelas populadas, você irá criar consultas SQL cada vez mais elaboradas, divididas por níveis (WHERE, JOIN, GROUP BY, etc.).
 - Ao final dessa etapa, elabore um documento explicando o que cada consulta recupera.
- **Parte 4 – PL/SQL:** Serão criadas funções, procedimentos e gatilhos para automatizar regras e processos dentro do banco.
 - Reutilize os dados da Parte 3 e organize seu código de forma modular.

Preparação do ambiente

1. **Verifique o container Docker correspondente ao banco de dados** (ex: Oracle, PostgreSQL, MySQL). Se ele não estiver em execução, solicite apoio para iniciar o container correto.
2. **Abra o DBeaver** e configure uma nova conexão, se necessário:
 - Vá em Arquivo > Nova Conexão > [Escolha o SGBD];
 - Preencha com os dados:
 - **Host:** localhost;
 - **Porta:** 1521 (Oracle), 5432 (PostgreSQL), 3306 (MySQL);
 - **Usuário e Senha:** conforme o ambiente de aula;
 - **Banco de Dados:** ex: labdatabase.
3. **Teste a conexão** e salve. A conexão aparecerá no painel esquerdo (Database Navigator).

Como usar o DBeaver de forma eficaz

- Utilize o editor de SQL no painel direito para escrever seus comandos.
- Execute scripts ou comandos selecionados com Ctrl + Enter.
- Use o painel lateral para visualizar e explorar os objetos do banco (tabelas, views, índices etc.).
- Organize seus scripts com comentários e indentação clara.

Atenção: Deep Freeze nas máquinas do laboratório

As máquinas da faculdade utilizam o software **Deep Freeze**, o que significa que **qualquer alteração ou arquivo salvo localmente será perdido ao reiniciar**.

Por isso, é obrigatório que você salve todos os seus scripts e arquivos em locais persistentes como:

- **GitHub** (recomendado para versionamento e portfólio);
- **Pendrive pessoal;**
- **Google Drive, Dropbox, OneDrive;**
- **E-mail pessoal (como anexo de segurança, se necessário).**

Organização recomendada dos scripts

Salve seus arquivos de forma clara e reutilizável. Exemplo de estrutura:

banco_de_dados/

├── 01_ddl/

| ├── criar_tabelas.sql

├── 02_dml/

| ├── inserir_dados.sql

| ├── atualizar_dados.sql

| └── excluir_dados.sql

├── 03_consultas/

| └── select_nivel_1_a_nivel_6.sql

├── 04_plsql/

| ├── funcoes.sql

| ├── procedures.sql

| └── triggers.sql

└── scripts_python/

| └─ executa_scripts.py

└─ README.md

Integração com Python

Ao longo do roteiro, você utilizará Python para automatizar a execução de scripts SQL. Isso simula integrações reais entre aplicações e bancos de dados.

- Os scripts devem ser salvos com extensão .py e executados via terminal ou VS Code.
- Os exemplos utilizam a classe OracleQueries, mas podem ser adaptados a outros SGBDs.

Parte 1 - DDL

1 - Create

A instrução CREATE é utilizada para criar as estruturas dentro do banco de dados. Podem ser criadas TABLES (TABELAS), VIEWS (Visões sobre uma ou um conjunto de tabelas ou views), INDEX (Índices), SEQUENCES (sequências autoincrementais controladas pelo banco de dados) e DATABASE (banco de dados).

Nesse momento, o banco de dados Lab Database está vazio e sempre que a máquina virtual iniciar esse será seu estado inicial, para que possamos ter um ambiente integro para as atividades serem desenvolvidas.

1.1 Tables

1. Sintaxe da instrução **Create Table** e exemplo de utilização:

Sintaxe	Exemplo
<pre>CREATE TABLE <table_name> (<column_name> <datatype> {null Not null} [...]) -- Constraints de tabela (opcionais) [CONSTRAINT {pk_name} PRIMARY KEY ({col1}[, {colN}]]], [CONSTRAINT {uq_name} UNIQUE ({col1}[, {colN}]]], [CONSTRAINT {fk_name} FOREIGN KEY ({col}) REFERENCES {ref_schema?}.{ref_table} ({ref_col})]); -- Comentários (quando suportado) [COMMENT ON TABLE {schema?}.{table_name} IS '{texto}']; -- Oracle/PG [COMMENT ON COLUMN {schema?}.{table_name}.{col} IS '{texto}']; -- Oracle/PG</pre>	<pre>CREATE TABLE CIDADE (CODIGO_CIDADE NUMBER(10) GENERATED BY DEFAULT AS IDENTITY NOT NULL, NOME_CIDADE VARCHAR2(100) NOT NULL, FUNDACAO DATE, CONSTRAINT PK_CIDADE PRIMARY KEY (CODIGO_CIDADE), CONSTRAINT UQ_CIDADE_NOME UNIQUE (NOME_CIDADE)); -- (Opcional) comentários COMMENT ON TABLE CIDADE IS 'Tabela de cidades'; COMMENT ON COLUMN CIDADE.CODIGO_CIDADE IS 'Identificador da cidade'; COMMENT ON COLUMN CIDADE.NOME_CIDADE IS 'Nome da cidade (único)'; COMMENT ON COLUMN CIDADE.FUNDACAO IS 'Data de fundação';</pre>

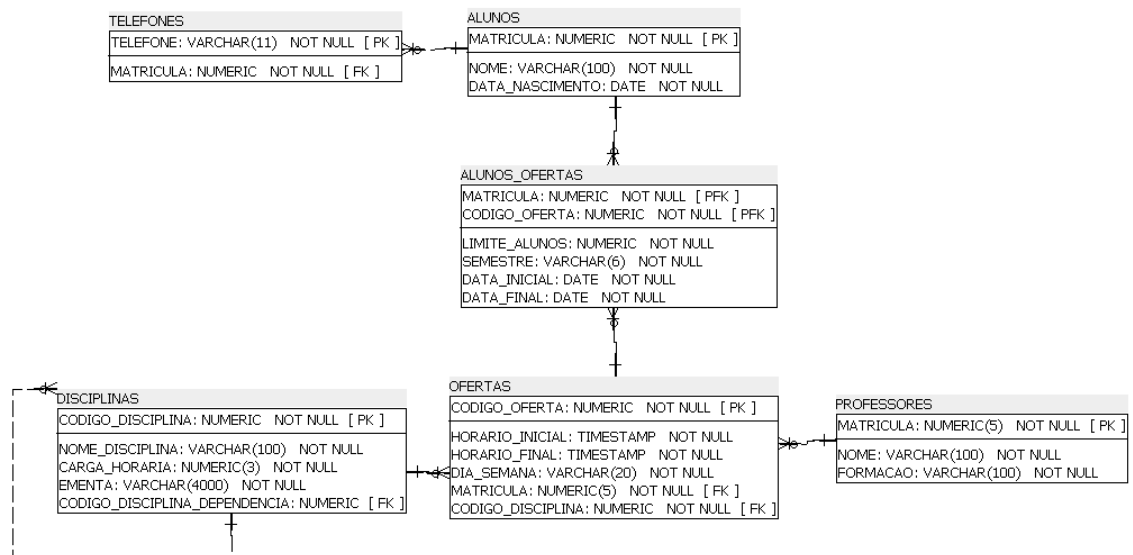
2. O Oracle fornece os seguintes tipos de dados built-in:

Oracle	PostgreSQL	MySQL	Observações / Notas
CHAR(n)	CHAR(n)	CHAR(n)	Todos são tamanho fixo. Em Oracle e MySQL, CHAR é preenchido com espaços; no PG também. Atenção a collation/charset (MySQL: defina utf8mb4).
VARCHAR2(n)	VARCHAR(n)	VARCHAR(n)	Portável. Em MySQL sempre use utf8mb4. Em Oracle/PG, VARCHAR/VARCHAR2 equivalem na prática.

Oracle	PostgreSQL	MySQL	Observações / Notas
NCHAR(n)	CHAR(n) (+ COLLATE se necessário)	NCHAR(n)	PG não tem NCHAR/NVARCHAR; use CHAR/VARCHAR com COLLATE/encoding do DB. Em MySQL, NCHAR usa “national character set” (em 8.0, geralmente utf8mb4).
NVARCHAR2(n)	VARCHAR(n) (+ COLLATE se necessário)	NVARCHAR(n)	Mesmo raciocínio do NCHAR. Em MySQL, NVARCHAR é sinônimo de VARCHAR com charset nacional (use utf8mb4).
CLOB	TEXT	LONGTEXT (ou TEXT/MEDIUMTEXT conforme o tamanho)	PG TEXT é prático (limite alto ~1 GB por valor). Em MySQL escolha a “família TEXT” conforme o volume (até 4 GB em LONGTEXT).
NCLOB	TEXT (+ COLLATE/encoding)	LONGTEXT (com utf8mb4)	Mesmo que CLOB, só que “national”. Garanta collations/charset corretos.
LONG (obsoleto)	TEXT	LONGTEXT	Tipo legado do Oracle; ao migrar, trate como texto grande.
NUMBER(p,s)	NUMERIC(p,s) (ou INTEGER/BIGINT se inteiro)	DECIMAL(p,s) (ou INT/BIGINT se inteiro)	Para inteiros puros, mapeie para tipos inteiros nativos. NUMERIC (PG) aceita precisão alta; DECIMAL (MySQL) até 65 dígitos.
DATE (data+hora)	TIMESTAMP (ou TIMESTAMP(0))	DATETIME (ou TIMESTAMP)	Em Oracle DATE inclui hora; em PG/MySQL DATE é só data. Para manter semântica, use TIMESTAMP (PG) e DATETIME/TIMESTAMP (MySQL).
BLOB	BYTEA	LOBLOB (ou BLOB/MEDIUMBLOB)	Em PG use BYTEA. Em MySQL escolha a “família BLOB” pelo tamanho esperado.
BFILE	(sem equivalente nativo) TEXT/VARCHAR para URL/caminho	(sem equivalente nativo) TEXT/VARCHAR para URL/caminho	BFILE aponta para arquivo externo; modele como caminho/URL + política de storage externa.
RAW(n)	BYTEA (+ CHECK para limitar tamanho)	VARBINARY(n)	PG não tem VARBINARY(n); use BYTEA e, se quiser, um CHECK (octet_length(col) ≤ n).
LONG RAW	BYTEA	LOBLOB	Tipo legado de binário variável; migrar como binário grande.

Oracle	PostgreSQL	MySQL	Observações / Notas
ROWID	(sem tipo equivalente) TEXT/VARCHAR(18)	VARCHAR(18)	ROWID é endereço físico Oracle. Em PG há ctid (system column, não estável). Armazene como texto apenas se precisar manter referência legada.
MLSLABEL	TEXT/JSONB (conforme uso)	JSON/TEXT	Tipo do Trusted Oracle; modele como TEXT ou estrutura (JSONB no PG) conforme a aplicação.

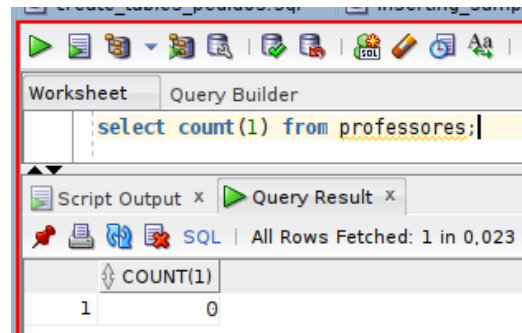
3. No painel de inclusão e execução de comandos, crie as tabelas do diagrama relacional (lógico) a seguir:



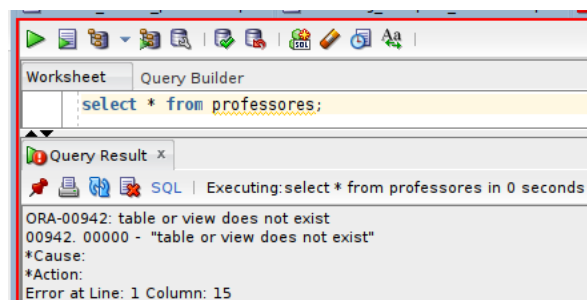
- a) **Dica:** analise as tabelas e verifique quais são independentes e quais são dependentes, devemos criar as tabelas independentes primeiro;
4. Nesse momento você não precisa se preocupar com os relacionamentos, mas caso já queira criá-los vá para a [Seção 2 - Alter Table](#)
5. Verifique se as tabelas foram criadas analisando o painel esquerdo, no diretório **Tables** ou execute no **SQL Developer** as seguintes consultas de contagem de registros no painel do lado direito:
- select** count(1) **from** telefones;
 - select** count(1) **from** alunos;

- c. **select** count(1) **from** alunos_ofertas;
- d. **select** count(1) **from** ofertas;
- e. **select** count(1) **from** professores;
- f. **select** count(1) **from** disciplinas;

6. Se as tabelas tiverem sido criadas corretamente, deverá retornar uma tela parecida com a imagem a seguir:



7. Caso as tabelas não tenham sido criadas corretamente, você irá tomar o seguinte erro conforme imagem a seguir:



- 8. Lembre-se que todas as declarações SQL terminam com ; (ponto e vírgula)
- 9. Agora, experimente alterar o script **Python** a seguir, utilizando o **VS Code** ou outra IDE de seu interesse, para executar a criação de suas tabelas automaticamente através do Python. Dado que você já as criou, será necessário apagá-las antes, consulte a [Seção 3 - Drop](#) e crie os comandos corretamente antes dos comandos de criação.
 - a. **Dica 1:** a exclusão das tabelas também precisa obedecer a uma ordem lógica, nesse caso, primeiro excluimos as tabelas dependentes e em seguida as tabelas independentes;

- b. **Dica 2:** para a execução do script Python a seguir, você precisará do [módulo conexão](#) que se encontra no [Github](#), basta baixar o projeto por completo e copiar o módulo para um novo projeto;
- c. **Dica 3:** salve as instruções de criação de tabela em um arquivo com a extensão .sql;

```
from connexion.oracle_queries import OracleQueries

def create_tables(query:str):
    list_of_commands = query.split(";")

    oracle = OracleQueries(can_write=True)
    oracle.connect()

    for command in list_of_commands:
        if len(command) > 0:
            print(command)
            try:
                oracle.executeDDL(command)
                print("Successfully executed")
            except Exception as e:
                print(e)

def run():
    with open("insira aqui o caminho para o script SQL criado") as f:
        query_create = f.read()

    print("Creating tables...")
    create_tables(query=query_create)
    print("Tables successfully created!")

if __name__ == '__main__':
    run()
```

1.2 View

Uma visão é um objeto do banco de dados que permite manter armazenada uma consulta mais elaborada ou complexa, para que não seja necessária sua criação constantemente. Além disso, uma view pode ser utilizada para manter seguro dados que tenham algum grau de sigilo.

1. Sintaxe da instrução **Create View** e exemplo de utilização:

Sintaxe	Exemplo
CREATE [OR REPLACE] VIEW <view_name> [(<column_name>, ...)] AS (SELECT statement))	CREATE OR REPLACE VIEW PEDIDOS_FORNECEDORES AS (WITH SUMARIZA_PEDIDOS AS (SELECT CNPJ , COUNT(1) AS QTD_PEDIDOS

	<pre> FROM PEDIDOS GROUP BY CNPJ) SELECT NVL(F.NOME_FANTASIA, F.RAZAO_SOCIAL) AS EMPRESA , SP.QTD_PEDIDOS , SUM(I.QUANTIDADE * I.VALOR_UNITARIO) AS VALOR_TOTAL FROM PEDIDOS P INNER JOIN SUMARIZA_PEDIDOS SP ON P.CNPJ = SP.CNPJ INNER JOIN FORNECEDORES F ON P.CNPJ = F.CNPJ INNER JOIN ITENS_PEDIDO I ON P.CODIGO_PEDIDO = I.CODIGO_PEDIDO GROUP BY SP.QTD_PEDIDOS , NVL(F.NOME_FANTASIA , F.RAZAO_SOCIAL)); </pre>
--	---

2. No painel de inclusão e execução de comandos, crie as views a seguir

NOME DA VIEW: DISCIPLINAS_PROFESSOR

QUERY PARA A VIEW:

```

SELECT P.NOME AS PROFESSOR
, P.FORMACAO
, D.CODIGO_DISCIPLINA
, D.NOME_DISCIPLINA
, D.CARGA_HORARIA
FROM OFERTAS O
INNER JOIN PROFESSORES P
ON O.MATRICULA = P.MATRICULA
INNER JOIN DISCIPLINAS D
ON O.CODIGO_DISCIPLINA = D.CODIGO_DISCIPLINA

```

NOME DA VIEW: ALUNOS_MATRICULADOS

QUERY PARA A VIEW:

```

SELECT A.MATRICULA
, A.NOME AS ALUNO
, AO.SEMESTRE
, O.HORARIO_INICIAL
, O.HORARIO_FINAL
, O.DIA_SEMANA
, P.NOME AS PROFESSOR
, D.NOME_DISCIPLINA

```

```

FROM ALUNOS_OFERTAS AO
INNER JOIN ALUNOS A
ON AO.MATRICULA = A.MATRICULA
INNER JOIN OFERTAS O
ON AO.CODIGO_OFERTA = O.CODIGO_OFERTA
INNER JOIN PROFESSORES P
ON O.MATRICULA = P.MATRICULA
INNER JOIN DISCIPLINAS D
ON O.CODIGO_DISCIPLINA = D.CODIGO_DISCIPLINA

```

3. Verifique se as views foram criadas analisando o painel esquerdo, no diretório Views
4. Experimente alterar a sugestão de script Python da **Seção 1 – Create, Item 8** e criar as views a partir dele também, será necessário apagá-las antes, consulte a [Seção 3 - Drop](#)
 - a. **Dica:** no mesmo script .sql que gerou na etapa anterior, inclua os comandos de exclusão e criação da view. Coloque as exclusões no início do script;

1.3 Index

Um índice é um objeto do banco de dados que permite o controle de dados de algumas colunas, auxiliando o banco de dados na recuperação acelerada dos dados de uma tabela.

1. Sintaxe da instrução **Create Index** e exemplo de utilização:

Sintaxe	Exemplo
CREATE [unique] INDEX <index_name> ON <table_name> (<column_name> [,...]) ;	CREATE INDEX CLIENTES_IDX ON CLIENTES (NOME);

2. No painel de inclusão e execução de comandos, crie os indexes a seguir
NOME DO ÍNDICE: ALUNOS_NOME_IDX
NOME DA TABELA: ALUNOS

NOME DA COLUNA: NOME

NOME DO ÍNDICE: OFERTAS_DIA_IDX

NOME DA TABELA: OFERTAS

NOME DA COLUNA: DIA_SEMANA

NOME DO ÍNDICE: DISCIPLINA_NOME_IDX

NOME DA TABELA: DISCIPLINA

NOME DA COLUNA: NOME_DISCIPLINA

NOME DO ÍNDICE: PROFESSORES_NOME_IDX

NOME DA TABELA: PROFESSORES

NOME DA COLUNA: NOME

3. Verifique se os índices foram criados analisando o painel esquerdo, no diretório Indexes
5. Experimente alterar a sugestão de script Python da **Subseção 1 – Item 9** e criar os índices a partir dele também, será necessário apagá-los antes, consulte a [Seção 3 - Drop](#)
 - a. **Dica:** no mesmo script .sql gerado anteriormente, inclua os comandos de exclusão e criação de índices;

1.4 Sequences

Uma *sequence* é um objeto sequencialmente incremental controlado pelo banco de dados. É uma espécie de *autoincrement* existente em outros bancos de dados. Para recuperar o próximo valor de uma sequência, basta utilizar o nome do objeto seguido do atributo **nextval**. Para recuperar o valor atual, basta utilizar o nome do objeto seguido do atributo **currentval**.

1. Sintaxe da instrução **Create Sequence** e exemplo de utilização:

Sintaxe	Exemplo
CREATE SEQUENCE <sequence_name>;	CREATE SEQUENCE fornecedores_seq;

2. Crie as *sequences* conforme orientações a seguir

NOME DA SEQUENCE: DISCIPLINAS_SEQ

NOME DA SEQUENCE: OFERTAS_SEQ

3. Verifique no painel esquerdo do SQL Developer clicando sobre o diretório *Sequences* as *sequences* que acaba de criar;
4. Experimente alterar a sugestão de script Python da Subseção 1 – Item 9 e criar as sequências a partir dele também, será necessário apagá-las antes, consulte a [Seção 3 - Drop](#)
 - a. **Dica:** no mesmo script .sql gerado anteriormente, inclua os comandos de exclusão e criação de sequências;

2 - Alter Table

Para modificar a estrutura das tabelas você irá utilizar o comando ***Alter Table***, com ela é possível: [adicionar uma ou mais colunas](#), [modificar a definição da coluna](#), [remover uma ou mais colunas](#), [renomear colunas](#), [renomear tabelas](#), [criar chave primária \(PK\)](#), [criar restrição de integridade \(FK\)](#), entre outros.

2.1 - Add Column

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
ALTER TABLE <table_name> ADD <column_name> <type> [<constraint>];	ALTER TABLE veiculos ADD placa VARCHAR2(10); ALTER TABLE seguros ADD apolice VARCHAR2(15) NOT NULL;

2. Adicione as colunas a seguir às devidas tabelas:

TABELA: ALUNOS

CAMPOS: EMAIL VARCHAR2(200) e CPF VARCHAR2(11)

RESTRIÇÃO: PERMITE NULL

TABELA: PROFESSORES

CAMPOS: CONTATO VARCHAR2(200)

RESTRIÇÃO: PERMITE NULL

TABELA: OFERTAS

CAMPOS: DATA_CRIACAO DATE

RESTRIÇÃO: PERMITE NULL

3. Verifique no painel esquerdo do SQL Developer clicando duas vezes sobre o nome da tabela a estrutura alterada;

4. Incremente seu script SQL com as instruções geradas;

2.2 - Modify Column

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
<code>ALTER TABLE <table_name> MODIFY <column_name> <type> [<constraint>];</code>	<code>ALTER TABLE veiculos MODIFY placa VARCHAR2(8) NOT NULL;</code>

2. Modifique as colunas das tabelas a seguir

TABELA: ALUNOS

CAMPOS: EMAIL VARCHAR2(250)

RESTRIÇÃO: PERMITE NULL

TABELA: OFERTAS

CAMPOS: DATA_CRIACAO DATE

RESTRIÇÃO: NÃO PERMITE NULL

DEFAULT: SYSDATE

3. Verifique no painel esquerdo do SQL Developer clicando duas vezes sobre o nome da tabela a estrutura alterada
4. Incremente seu script SQL com as instruções geradas;

2.3 - Rename Column

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
<code>ALTER TABLE <table_name> RENAME COLUMN <old_name> TO <new_name>;</code>	<code>ALTER TABLE fornecedores RENAME COLUMN nome TO nomefantasia;</code>

2. Renomeie as colunas das tabelas a seguir

TABELA: ALUNOS

NOME ANTIGO: MATRICULA, NOME

NOVO NOME: MATRICULA_ALUNO, NOME_ALUNO

TABELA: PROFESSORES

NOME ANTIGO: MATRICULA, NOME

NOVO NOME: MATRICULA_PROFESSOR, NOME_PROFESSOR

TABELA: DISCIPLINAS
NOME ANTIGO: EMENTA
NOVO NOME: EMENTA_DISCIPLINA

TABELA: ALUNOS_OFERTAS
NOME ANTIGO: MATRICULA
NOVO NOME: MATRICULA_ALUNO

TABELA: OFERTAS
NOME ANTIGO: MATRICULA
NOVO NOME: MATRICULA_PROFESSOR

3. Verifique no painel esquerdo do SQL Developer clicando duas vezes sobre o nome da tabela a estrutura alterada;
4. Incremente seu script SQL com as instruções geradas;

2.4 - Rename Table

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
ALTER TABLE <table_name> RENAME TO <new_table name>;	ALTER TABLE fornecedores RENAME TO provedores;

2. Renomeie a tabela conforme instruções a seguir
NOME ANTIGO: TELEFONES
NOVO NOME: TELEFONES_ALUNOS
3. Verifique no painel esquerdo do SQL Developer clicando no botão de atualizar o novo nome da tabela;
4. Incremente seu script SQL com as instruções geradas;

2.5 - Drop Column

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
ALTER TABLE <table_name> DROP COLUMN <column_name>;	ALTER TABLE veiculos DROP COLUMN placa;

2. Remova a coluna conforme instruções a seguir

TABELA: PROFESSORES

COLUNA: CONTATO

TABELA: ALUNOS

COLUNA: EMAIL

TABELA: ALUNOS

COLUNA: CPF

3. Verifique no painel esquerdo do SQL Developer clicando duas vezes no nome da tabela a alteração realizada;
4. Incremente seu script SQL com as instruções geradas;

2.6 - Add Primary Key

A criação das chaves primárias (PK) e estrangeiras (FK) pode ser feitas de diversas formas distintas. Pode ser criado ao mesmo tempo da criação da tabela ou utilizando a instrução ALTER TABLE. Nessa etapa iremos utilizar o ALTER TABLE, mas você está sendo encorajado a pesquisar a forma de [criação de chaves primárias e estrangeiras através do comando CREATE TABLE](#).

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
ALTER TABLE <table_name> ADD [CONSTRAINT <pk_name>] PRIMARY KEY (<identification_column_name>);	ALTER TABLE fornecedores ADD PRIMARY KEY (CNPJ);

2. Adicione as chaves primárias das tabelas de acordo com as orientações a seguir:

TABELA: TELEFONES

CAMPO: TELEFONE

TABELA: ALUNOS

CAMPO: MATRICULA_ALUNO

TABELA: ALUNOS_OFERTAS

CAMPO: MATRICULA_ALUNO, CODIGO_OFERTA

TABELA: OFERTAS

CAMPO: CODIGO_OFERTA

TABELA: PROFESSORES

CAMPO: MATRICULA_PROFESSOR

TABELA: DISCIPLINAS

CAMPO: CODIGO_DISCIPLINA

3. Verifique no painel esquerdo do SQL Developer clicando duas vezes no nome da tabela a alteração realizada;
4. Incremente seu script SQL com as instruções geradas;

2.7 - Add Constraint

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
<pre>ALTER TABLE <table_name> ADD CONSTRAINT <constraint_name> FOREIGN KEY (column1, column2, ..., column_n) REFERENCES <parent_table> (column1, column2, ..., column_n);</pre>	<pre>ALTER TABLE PEDIDOS ADD CONSTRAINT FORNECEDORES_PEDIDOS_FK FOREIGN KEY (CNPJ) REFERENCES FORNECEDORES (CNPJ);</pre>

2. Adicione as chaves estrangeiras conforme orientações a seguir

TABELA: TELEFONES_ALUNOS

CAMPO: MATRICULA_ALUNO

TABELA REFERÊNCIA: ALUNOS

COLUNA REFERÊNCIA: MATRICULA_ALUNO

NOME DA CONSTRAINT: TELEFONE_ALUNO_FK

TABELA: ALUNOS_OFERTAS

CAMPO: MATRICULA_ALUNO

TABELA REFERÊNCIA: ALUNOS

COLUNA REFERÊNCIA: MATRICULA_ALUNO

NOME DA CONSTRAINT: ALUNOS_ALUNO_FK

TABELA: ALUNOS_OFERTAS

CAMPO: CODIGO_OFERTA

TABELA REFERÊNCIA: OFERTAS

COLUNA REFERÊNCIA: CODIGO_OFERTA

NOME DA CONSTRAINT: ALUNOS_OFERTA_FK

TABELA: OFERTAS

CAMPO: MATRICULA_PROFESSOR

TABELA REFERÊNCIA: PROFESSORES

COLUNA REFERÊNCIA: MATRICULA_PROFESSOR

NOME DA CONSTRAINT: OFERTAS_PROFESSOR_FK

TABELA: OFERTAS

CAMPO: CODIGO_DISCIPLINA

TABELA REFERÊNCIA: DISCIPLINAS

COLUNA REFERÊNCIA: CODIGO_DISCIPLINA

NOME DA CONSTRAINT: OFERTAS_DISCIPLINAS_FK

TABELA: DISCIPLINAS

CAMPO: CODIGO_DISCIPLINA_DEPENDENCIA

TABELA REFERÊNCIA: DISCIPLINAS

COLUNA REFERÊNCIA: CODIGO_DISCIPLINA

NOME DA CONSTRAINT: DISCIPLINAS_DEPENDENCIA_FK

3. Verifique no painel esquerdo do SQL Developer clicando duas vezes no nome da tabela a alteração realizada;
4. Incremente seu script SQL com as instruções geradas;

3 - Drop

3.1 - Tables

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
DROP TABLE <table_name>;	DROP TABLE professores;

2. Remova todas as tabelas criadas na [Seção 1 - Create Subseção 1 - Tables](#)
3. Confirme que as tabelas foram removidas através do painel esquerdo do SQL Developer;
4. Incremente seu script SQL com as instruções geradas;

3.2 - View

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
DROP VIEW <view_name>;	DROP VIEW pedidos_fornecedores;

2. Remova todas as views criadas na [Seção 1 - Create Subseção 2 - Views](#)
3. Confirme que as views foram removidas através do painel esquerdo do SQL Developer;
4. Incremente seu script SQL com as instruções geradas;

3.3 - Index

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
DROP INDEX <index_name>;	DROP INDEX clientes_idx;

2. Remova todos os índices criados na [Seção 1 - Create Subseção 3 - Index](#)
3. Confirme que os índices foram removidas através do painel esquerdo do SQL Developer;
4. Incremente seu script SQL com as instruções geradas;

3.4 - Constraints

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
ALTER TABLE <table_name> DROP CONSTRAINT <constraint_name>;	ALTER TABLE PEDIDOS DROP CONSTRAINT FORNECEDORES_PEDIDOS_FK;

2. Remova todas as constraints Foreign Key criadas na [Seção 2 – Alter Table Subseção 2.7 - Add Constraint](#)
3. Confirme que as *constraints* foram removidas através do painel esquerdo do SQL Developer;
4. Incremente seu script SQL com as instruções geradas;

3.5 - Sequences

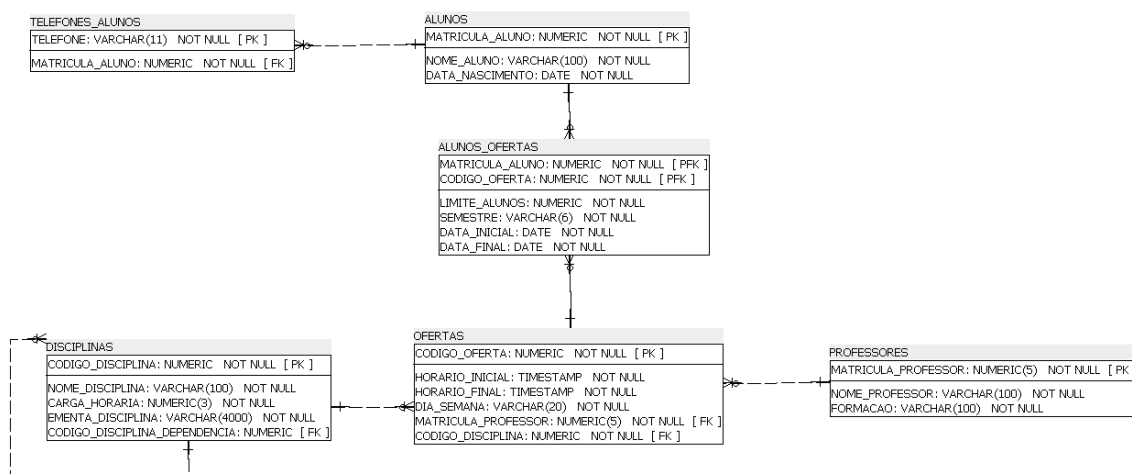
1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
<code>DROP SEQUENCE <sequence_name>;</code>	<code>DROP SEQUENCE PEDIDOS_CODIGO_PEDIDO_SEQ;</code>

2. Remova todas as *sequences* criadas na [Seção 1 - Create Subseção 2.8 - Sequences](#)
3. Confirme que as *sequences* foram removidas através do painel esquerdo do SQL Developer;
4. Incremente seu script SQL com as instruções geradas;

Seção Final da Parte 1

Espera-se, ao final dessa prática, que você obtenha a seguinte estrutura de dados:



Como tarefa final, desenvolva um script SQL que permita apagar todos os objetos e recriá-los, conforme exemplo disponível em: https://github.com/howardroatti/example_crud_oracle/blob/master/sql/create_tables_pedidos.sql. **Esse script é fundamental para a continuação desse roteiro.**

Após criar o seu script SQL, altere o trecho de código Python a seguir e o execute com seu script SQL:

```
from connexion.oracle_queries import OracleQueries

def execute_commands(query:str):
    list_of_commands = query.split(";")

    oracle = OracleQueries(can_write=True)
    oracle.connect()

    for command in list_of_commands:
        if len(command) > 0:
            print(command)
            try:
                oracle.executeDDL(command)
                print("Successfully executed")
            except Exception as e:
                print(e)

def run():

    with open("insira aqui o caminho para o seu script") as f:
        query_create = f.read()

    print("Creating tables...")
    execute_commands(query=query_create)
    print("Tables successfully created!")

if __name__ == '__main__':
    run()
```

Parte 2 - DML

Execute o Script criado na [Parte 1](#) desse roteiro. Então, siga para as próximas seções.

Seção 1 - Inserção

1. Na ordem em que as tabelas aparecem, insira as respectivas linhas:

ALUNOS	MATRICULA_ALUNO	NOME_ALUNO	DATA_NASCIMENTO
	40001	JOÃO GABRIEL	02/04/1985
	40002	JOÃO JOSÉ	31/12/2001
	40003	JOSÉ GABRIEL	04/07/1999
	40004	ANTONIO JOSÉ	27/09/1998
	40005	ANTONIO MARCIO	20/04/1955
	40006	JOSÉ ANTÔNIO	29/04/1955
	40007	MARCIO ANTONIO	14/03/1967
	40008	JOÃO ANTONIO	21/02/2002
	40009	MARCIO GABRIEL	08/06/2009
	40010	GABRIEL ANTONIO	01/01/1970

TELEFONES_ALUNOS	TELEFONE	MATRICULA_ALUNO
	11990897030	40001
	27990897031	40001
	33990897032	40002
	22990897033	40010
	27990897034	40003
	22990897035	40006
	11990897036	40008
	14990897037	40003
	18990897038	40004
	79990897039	40007
	83990897040	40005
	81990897041	40009

PROFESSORES	MATRICULA_PROFESSOR	NOME_PROFESSOR	FORMACAO
	55124	CARLOS ANTONIO	MESTRADO
	55125	PAULO ANTONIO	DOUTORADO
	55126	PATRICIA CARMEN	ESPECIALIZAÇÃO
	55127	ALINE SANTOS	MESTRADO
	55128	ALICE DE SOUZA	MESTRADO
	55129	LUIZ CARLOS	DOUTORADO
	55130	LUCAS PEDRO	DOUTORADO
	55131	PEDRO HENRIQUE	MESTRADO
	55132	MARIA OLGA	DOUTORADO
	55133	CAMILA PEREZ	ESPECIALIZAÇÃO

DISCIPLINAS	CODIGO_DISCIPLINA	NOME_DISCIPLINA	CARGA_HORARIA	EMENTA_DISCIPLINA	CODIGO_DISCIPLINA_DEPENDENCIA
	48851	DATA SCIENCE	80	EMENTA DA DISCIPLINA DATA SCIENCE	75189
	45812	DATA ANALYTICS	40	EMENTA DA DISCIPLINA DATA ANALYTICS	
	33412	MACHINE LEARNING	80	EMENTA DA DISCIPLINA MACHINE LEARNING	
	12345	NATURAL LANGUAGE PROCESSING	80	EMENTA DA DISCIPLINA NATURAL LANGUAGE PROCESSING	33412
	55213	COMPUTER VISION	80	EMENTA DA DISCIPLINA COMPUTER VISION	33412
	18493	ROBOTIC	60	EMENTA DA DISCIPLINA ROBOTIC	31313
	13131	DATABASE	60	EMENTA DA DISCIPLINA DATABASE	31313
	31313	COMPUTER PROGRAMMING	30	EMENTA DA DISCIPLINA COMPUTER PROGRAMMING	
	85853	BIG DATA	75	EMENTA DA DISCIPLINA BIG DATA	75189
	75189	DATA MINING	80	EMENTA DA DISCIPLINA DATA MINING	

OFERTAS	CODIGO_OFERTA	HORARIO_INICIAL	HORARIO_FINAL	DIA_SEMANA	MATRICULA_PROFESSOR	CODIGO_DISCIPLINA
	1418	18:45	22:00	SEGUNDA	55125	33412
	1519	20:30	22:00	SEGUNDA	55131	12345
	1742	18:45	20:15	TERÇA	55125	48851
	1112	19:00	23:00	QUARTA	55126	55213
	1313	17:00	21:00	QUARTA	55130	13131
	1632	18:30	20:00	SEXTA	55125	12345
	1674	19:30	21:30	QUINTA	55133	55213

ALUNOS_OFERTAS	MATRICULA_ALUNO	CODIGO_OFERTA	LIMITE_ALUNOS	SEMESTRE	DATA_INICIAL	DATA_FINAL
	40001	1418	40	2022/2	01/07/2022	16/12/2022
	40001	1519	20	2022/2	01/07/2022	16/12/2022
	40001	1313	35	2022/2	01/07/2022	16/12/2022
	40002	1418	40	2022/2	01/07/2022	16/12/2022
	40002	1632	20	2022/2	01/07/2022	16/12/2022
	40003	1418	40	2022/2	01/07/2022	16/12/2022
	40003	1519	20	2022/2	01/07/2022	16/12/2022
	40004	1418	40	2022/2	01/07/2022	16/12/2022
	40004	1313	20	2022/2	01/07/2022	16/12/2022
	40005	1418	40	2022/2	01/07/2022	16/12/2022
	40006	1418	40	2022/2	01/07/2022	16/12/2022
	40006	1674	20	2022/2	01/07/2022	16/12/2022
	40006	1519	35	2022/2	01/07/2022	16/12/2022
	40007	1674	40	2022/2	01/07/2022	16/12/2022
	40007	1313	20	2022/2	01/07/2022	16/12/2022
	40008	1313	40	2022/2	01/07/2022	16/12/2022
	40008	1519	20	2022/2	01/07/2022	16/12/2022
	40009	1674	40	2022/2	01/07/2022	16/12/2022
	40010	1418	40	2022/2	01/07/2022	16/12/2022

2. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
INSERT INTO <table_name> [(column1, column2, ..., column_n)] VALUES (value1, value2, ..., value3);	INSERT INTO fornecedores (cnpj, razaosocial, nomefantasia, capital, data_fundacao) VALUES ('1234567890001', 'Certeza de Vendas LTDA', 'Vendas Certas', 450289.33, TO_DATE ('01/01/1900', 'DD/MM/YYYY'))); INSERT INTO fornecedores VALUES ('1234567890001', 'Certeza de Vendas LTDA', 'Vendas Certas', 450289.33, TO_DATE ('01/01/1900', 'DD/MM/YYYY'))); INSERT INTO fornecedores (cnpj, razaosocial, nomefantasia, capital, data_fundacao) SELECT cnpj, razaosocial, nomefantasia, capital, data_fundacao FROM fornecedores_ativos;

3. Após realizar todas as inclusões, certifique-se de confirmar suas alterações executando o comando a seguir, caso contrário, seus dados não serão persistidos:

COMMIT;

Obs.: Caso queira desfazer as alterações, execute o comando **ROLLBACK;**

4. Verifique os registros inseridos através do painel à esquerda do **SQL Developer** clicando sobre a tabela a qual os dados foram inseridos ou execute as consultas descritas no Item 5 de 1.1 - Tables
5. Crie um Script SQL com todas as instruções de inserção de dados para que possa reutilizá-los posteriormente, use como exemplo o seguinte script SQL
- https://github.com/howardroatti/example_crud_oracle/blob/master/sql/inserting_samples_records.sql;
6. Dando continuidade ao script Python da Seção Final da Parte 1, insira o caminho para o script de inserção no local adequado e experimente executar o script Python para criação das tabelas, relacionamentos e registros:

```

from connexion.oracle_queries import OracleQueries

def create_tables(query:str):
    list_of_commands = query.split(";")

    oracle = OracleQueries(can_write=True)
    oracle.connect()

    for command in list_of_commands:
        if len(command) > 0:
            print(command)
            try:
                oracle.executeDDL(command)
                print("Successfully executed")
            except Exception as e:
                print(e)

def generate_records(query:str, sep:str=';'):
    list_of_commands = query.split(sep)

    oracle = OracleQueries(can_write=True)
    oracle.connect()

    for command in list_of_commands:
        if len(command) > 0:
            print(command)
            oracle.write(command)
            print("Successfully executed")

def run():
    with open("insira aqui o caminho para o seu script de criação") as f:
        query_create = f.read()

        print("Creating tables...")
        create_tables(query=query_create)
        print("Tables successfully created!")

    with open("insira aqui o caminho para o seu script de inserção")
as f:
        query_generate_records = f.read()

        print("Gerenerating records")
        generate_records(query=query_generate_records)
        print("Records successfully generated!")

if __name__ == '__main__':
    run()

```

Seção 2 - Atualização

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
<pre>UPDATE <table_name> SET <column1> = <value1>[, <column2> = <value2>, ... <column_n> = <value_n>] [WHERE <conditions statements>];</pre>	<pre>UPDATE CLIENTES SET NOME = 'NOVO NOME' WHERE CPF = '12345678911'; UPDATE ITENS_PEDIDO SET QUANTIDADE = 42, VALOR_UNITARIO = 17.99, CODIGO_PEDIDO = 18, CODIGO_PRODUTO = 43 WHERE CODIGO_ITEM_PEDIDO = 18; UPDATE SERVICIO SET VALOR = VALOR + VALOR * 0.3 WHERE COD_SERV IN (SELECT SF.COD_SERV FROM SERVICIO_FERRAMENTA SF INNER JOIN FERRAMENTA F ON SF.COD_FERR = F.COD_FERR WHERE UPPER(F.NOME) = 'MACACO HIDRÁULICO');</pre>

2. Realize as atualizações conforme detalhado a seguir

- Atualizar o atributo **DATA_NASCIMENTO** para **TO_DATE('29/04/2011', 'DD/MM/YYYY')** na tabela **ALUNOS** onde a **MATRICULA_ALUNO** seja igual a 40004;
- Atualizar o atributo **FORMACAO** para 'PHD' na tabela **PROFESSORES** onde a **FORMACAO** é 'DOUTORADO';
- Atualizar o atributo **FORMACAO** para 'MSC' na tabela **PROFESSORES** onde a **MATRICULA_PROFESSOR** seja 55126 e 55133;
- Atualizar o atributo **CODIGO_DISCIPLINA_DEPENDENCIA** para 31313 na tabela **DISCIPLINAS** onde o **NOME_DISCIPLINA** é 'MACHINE LEARNING';
- Atualizar o atributo **MATRICULA_PROFESSOR** para 55128 na tabela **OFERTAS** onde o **CODIGO_DISCIPLINA** seja 12345 e a **MATRICULA_PROFESSOR** seja 55125;
- Atualizar o atributo **LIMITE_ALUNOS** para 70 na tabela **ALUNOS_OFERTAS** onde a **OFERTA** é da **DISCIPLINA** 'NATURAL LANGUAGE PROCESSING';
- Faça a confirmação das alterações: **COMMIT;**

3. Após realizar todas as atualizações, certifique-se de confirmar suas alterações executando o comando a seguir, caso contrário, seus dados não serão persistidos:

COMMIT;

Obs.: Caso queira desfazer as alterações, execute o comando **ROLLBACK;**

4. Verifique os registros atualizados através do painel à esquerda do **SQL Developer** clicando sobre a tabela a qual os dados foram alterados ou execute uma das consultas a seguir antes e depois das atualizações:

- a. **select * from** telefones;
- b. **select * from** alunos;
- c. **select * from** alunos_ofertas;
- d. **select * from** ofertas;
- e. **select * from** professores;
- f. **select * from** disciplinas;

Seção 3 - Exclusão

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe	Exemplo
<code>DELETE FROM <table_name> [WHERE <conditions>];</code>	<code>DELETE FROM ITENS_PEDIDO; --MUITO CUIDADO COM ESSA INSTRUÇÃO!!!</code> <code>DELETE FROM FORNECEDORES WHERE NOME_FORNECEDOR = 'ALIADO';</code> <code>DELETE FROM FORNECEDORES WHERE NOME_FORNECEDOR LIKE 'AMIGOS%';</code> <code>DELETE FROM PRODUTOS WHERE CATEGORIA = 50 AND NOME_PRODUTO <> 'PERA';</code> <code>DELETE FROM PEDIDOS P WHERE EXISTS (SELECT * FROM CLIENTES C WHERE C.CPF = P.CPF AND C.NOME_CLIENTE = 'JOÃO JOSÉ');</code>

2. Realize a exclusão dos registros das tabelas conforme orientações a seguir:

- Exclua as disciplinas 85853 e 75189;
- Exclua a oferta 1742;
- Exclua os professores que não estão em alguma oferta;
- Exclua todos os telefones da que começam com 27;
- Desfaça as alterações: **ROLLBACK;**

3. Verifique os registros apagados através do painel à esquerda do SQL Developer clicando sobre a tabela a qual os dados foram apagados ou execute uma das consultas a seguir antes e depois das atualizações:

- select * from** telefones;
- select * from** alunos;
- select * from** alunos_ofertas;
- select * from** ofertas;
- select * from** professores;
- select * from** disciplinas;

Seção 4 - Consultas

1. Sintaxe da instrução e exemplo de utilização:

Sintaxe Básica	Sintaxe Avançada
<pre>SELECT [all distinct] select_list_columns FROM {table_name view_name} [, {table_name view_name} [WHERE search_conditions]</pre>	<pre>SELECT [all distinct] select_list_columns [INTO <variables>] FROM {table_name view_name} [, {table_name view_name} [INNER JOIN {<table_name> <view_name>} ON <conditions_to_join>] [WHERE <search_conditions>] [GROUP BY [all] column_name [, column_name] ...] [HAVING <search_conditions>] [ORDER BY {column_name list_number expression} [asc desc] [, {column_name list_number expression } [asc desc] ...]</pre>

2. Execute as consultas a seguir:

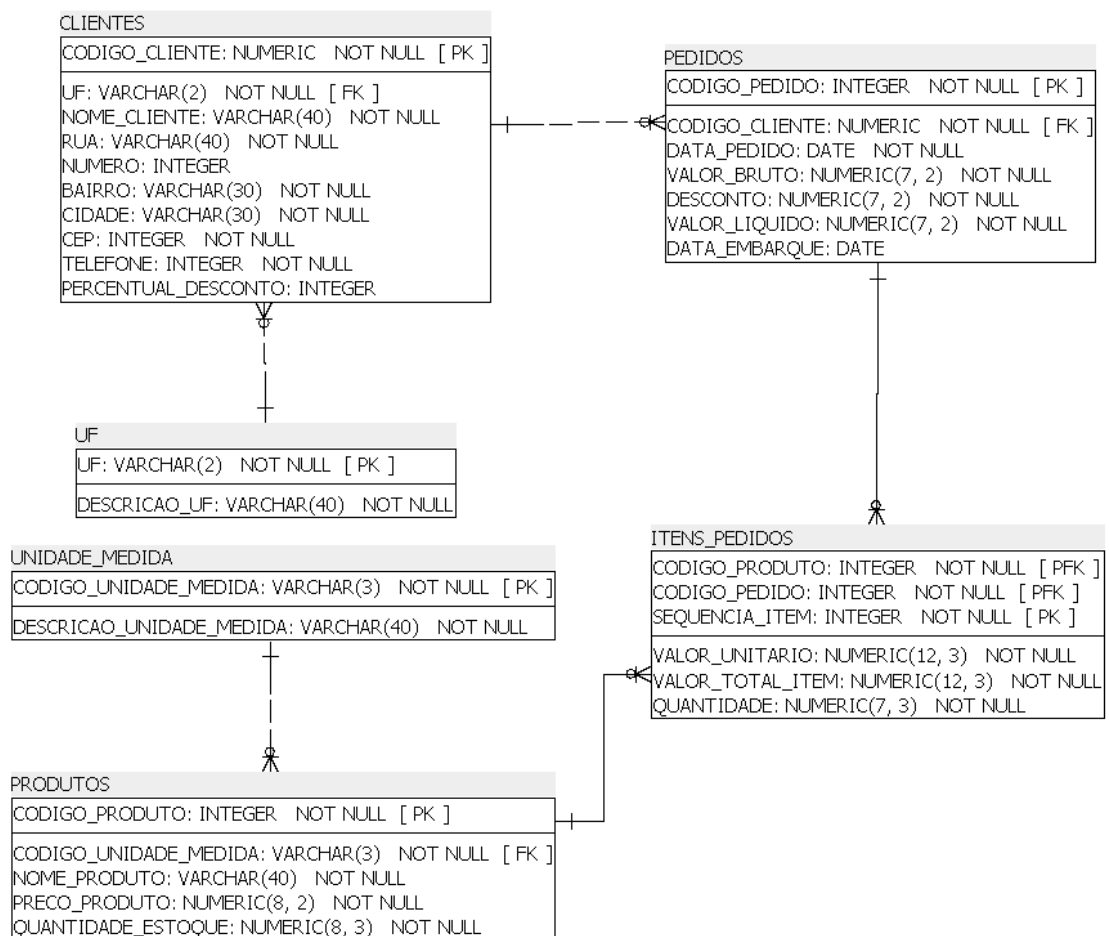
- SELECT * FROM ALUNOS;**
- ```
SELECT MATRICULA_ALUNO
 , INITCAP(NOME_ALUNO) AS NOME_ALUNO
 , TO_DATE(DATA_NASCIMENTO, 'DD/MM/YYYY') AS
DATA_NASCIMENTO
FROM ALUNOS
WHERE DATA_NASCIMENTO <= TO_DATE('01/01/2000', 'DD/MM/YYYY');
```
- ```
SELECT COUNT(1)
FROM ALUNOS
WHERE DATA_NASCIMENTO >= '01/01/2000';
```
- ```
SELECT A.NOME_ALUNO
 , T.TELEFONE
FROM ALUNOS A
INNER JOIN TELEFONES_ALUNOS T
ON A.MATRICULA_ALUNO = T.MATRICULA_ALUNO
ORDER BY A.NOME_ALUNO;
```
- ```
SELECT *
FROM ALUNOS
WHERE NOME_ALUNO LIKE '%ANTONIO%';
```
- ```
SELECT *
FROM PROFESSORES P
WHERE NOT EXISTS (SELECT 1
 FROM OFERTAS O
 WHERE O.MATRICULA_PROFESSOR =
P.MATRICULA_PROFESSOR);
```
- ```
SELECT *
FROM DISCIPLINAS
WHERE CODIGO_DISCIPLINA IN (85853, 75189);
```

- h. **SELECT ***
FROM DISCIPLINAS
WHERE CODIGO_DISCIPLINA_DEPENDENCIA IS NULL;
- i. **SELECT TO_CHAR(O.HORARIO_INICIAL, 'HH24:MI') AS HORARIO_INICIAL**
, TO_CHAR(O.HORARIO_FINAL, 'HH24:MI') AS HORARIO_FINAL
, P.NOME_PROFESSOR
FROM OFERTAS O
INNER JOIN PROFESSORES P
ON O.MATRICULA_PROFESSOR = P.MATRICULA_PROFESSOR
WHERE P.MATRICULA_PROFESSOR = 55125;
- j. **SELECT A.NOME_ALUNO**
, AO.SEMESTRE
, TO_CHAR(O.HORARIO_INICIAL, 'HH24:MI') AS HORARIO_INICIAL
, TO_CHAR(O.HORARIO_FINAL, 'HH24:MI') AS HORARIO_FINAL
, LOWER(O.DIA_SEMANA) AS DIA_SEMANA
, D.NOME_DISCIPLINA
, P.NOME_PROFESSOR
, D.CARGA_HORARIA
FROM ALUNOS_OFERTAS AO
INNER JOIN OFERTAS O
ON AO.CODIGO_OFERTA = O.CODIGO_OFERTA
INNER JOIN ALUNOS A
ON AO.MATRICULA_ALUNO = A.MATRICULA_ALUNO
INNER JOIN PROFESSORES P
ON O.MATRICULA_PROFESSOR = P.MATRICULA_PROFESSOR
INNER JOIN DISCIPLINAS D
ON O.CODIGO_DISCIPLINA = D.CODIGO_DISCIPLINA
WHERE D.CODIGO_DISCIPLINA_DEPENDENCIA IS NOT NULL
ORDER BY A.NOME_ALUNO;
- k. **SELECT D.NOME_DISCIPLINA AS PRE_REQUISITO**
, DD.NOME_DISCIPLINA AS DISCIPLINA
FROM DISCIPLINAS D
INNER JOIN DISCIPLINAS DD
ON D.CODIGO_DISCIPLINA = DD.CODIGO_DISCIPLINA_DEPENDENCIA
ORDER BY 1, 2;
- l. **SELECT COUNT(1) AS TOTAL_DISCIPLINAS**
, P.NOME_PROFESSOR
FROM OFERTAS O
INNER JOIN DISCIPLINAS D
ON O.CODIGO_DISCIPLINA = D.CODIGO_DISCIPLINA
INNER JOIN PROFESSORES P
ON O.MATRICULA_PROFESSOR = P.MATRICULA_PROFESSOR
GROUP BY P.NOME_PROFESSOR
ORDER BY P.NOME_PROFESSOR;

3. Descreva brevemente o que cada consulta recupera, detalhando as cláusulas existentes;

Parte 3 - DML – Queries (Select)

1. Agora é a sua vez, observe as sintaxes da instrução vistas ao final da [Seção 4 - Consultas](#) e então crie suas consultas de acordo com as orientações a seguir:
 - a. Baixe o script SQL da Unidade 5 denominado: **FAESA Banco de Dados V3.SQL**
 - b. Execute o script SQL via SQL Developer ou via Script Python, aproveita para exercitar
2. Modelo Relacional para auxiliar na criação das consultas:



3. Desenvolva as consultas solicitadas a seguir:

Nível 1 - Cláusula Where

- a. Selecionar todas as colunas de todos os clientes. **(40 linhas)**
- b. Selecionar somente a coluna nome de todos os produtos. **(47 linhas)**
- c. Selecionar as colunas “cidade”, “estado” e “CEP” de todos os clientes eliminando as redundâncias. **(32 linhas)**

- d. Selecionar todos os pedidos feitos pelo cliente cujo código seja = 04 e cujo valor total bruto seja maior que 10000. **(5 linhas)**
- e. Selecionar todos os pedidos cujo valor total bruto seja menor que 100 ou maior que 5000 e a data esteja compreendida entre 13/01/2007 e 14/01/2007. **(9 linhas)**
 - i. **Dica:** pesquise por TO DATE e pela cláusula BETWEEN
- f. Selecionar todos os clientes cujo código NÃO esteja compreendido entre 05 e 25. **(19 linhas)**
- g. Selecionar todos os produtos cujo nome possua a primeira letra = M, que possuam a string 'AN' em qualquer posição do nome, mas que terminem com a letra 'A'; **(2 linhas)**
 - i. **Dica:** pesquise por LIKE
- h. Selecionar todos os produtos que comecem com a letra MA e tenha exatamente 6 caracteres a mais. **(3 linhas)**
- i. Selecionar todos os produtos cujo nome possua a string 'ACA' a partir da terceira posição. **(1 linha)**
- j. Selecionar todos os Produtos cujo Nome possua a string 'A_P' como parte do nome do produto, em qualquer posição do nome. **(1 linha)**
 - i. **Dica:** pesquise por ESCAPE
- k. Selecionar todos os clientes cuja UF seja 'MG' ou 'ES'. **(8 linhas)**
 - i. **Dica:** pesquise por IN e NOT IN
- l. Selecionar todos os clientes cujo estado NÃO seja RJ nem SP. **(25 linhas)**

Nível 2 - Joins

- m. Selecionar o nome, o preço e a descrição da unidade de estoque dos produtos cujo preço seja menor que 20 e a descrição da unidade de estoque seja 'KILOGRAMA' ou 'LITRO', incluindo os produtos cujo preço seja nulo, usando aliases para colunas e tabelas. **(14 linhas)**
- n. Mostrar código do pedido, código e nome do cliente, código e nome do produto e descrição da unidade de estoque do produto. **(180 linhas)**

- o. Selecionar o código, o nome, o preço atual, e uma nova coluna exibindo o preço com um aumento 30% de todos os produtos que já tenham sido comprados, colocando ALIASES para coluna e tabela. **(39 linhas)**
 - i. Dica: Se você obteve um resultado superior a 39 linhas, analise a forma que fez a restrição sobre os produtos que já tenham sido comprados, experimente uma maneira diferente.

Nível 3 - Funções Agregadas

- p. Selecionar código do cliente, nome do cliente e valor total comprado por cada cliente, classificando por ordem descendente de valor total comprado usando aliases. **(18 linhas)**
 - i. Dica: pesquise por [SUM e GROUP BY](#), [ORDER BY DESC](#)
- q. Contar a quantidade de pedidos feitos para o cliente 05.
(Resultado: 2)
 - i. Dica: pesquise por [COUNT](#)
- r. Obter da tabela de pedidos o menor, o maior, a média e o somatório dos valores vendidos. **(Resultado: 15, 22540, 4898,7069..., 318416)**
 - i. Dica 1: pesquise por [MIN](#), [MAX](#) e [AVG](#)
 - ii. Dica 2: É comum o resultado de uma média de um valor com muitas casas decimais. Podemos melhorar a exibição desse resultado aplicação a função **ROUND**. Ex.:
ROUND(valor, numero de casas decimais).
- s. Mostrar o código de cada produto e a média de quantidade por produto vendido. **(39 linhas)**
 - i. **Dica:** Ordene o resultado por código do produto para melhor visualizar o resultado
- t. Mostrar o código de cada Unidade Federativa e a quantidade de clientes de cada unidade que já tenham feito pedido. **(Resultado: RJ=5, ES=3, SP=3, BA=3, RS=2, PR=1, SC=1)**
 - i. **Dica:** Se você obteve um resultado diferente do que está descrito acima (possivelmente com valores maiores),

observe a forma que fez a restrição para os clientes que já fizeram pedidos.

- u. Mostrar o código do cliente, e a média de valor comprado por cada cliente, somente para os clientes cuja média for maior que 8000. **(3 linhas)**
- v. Mostrar, o código, nome dos produtos e valor total vendido por produto, selecionando somente os produtos cujo valor total vendido for > 800, classificando por valor total vendido decedente. **(27 linhas)**
- w. Mostrar os dados dos produtos para os quais ainda não foi feito pedido. **(8 linhas)**
 - i. **Dica:** experimente as condicionais [IS NULL](#) e [IS NOT NULL](#)
- x. Mostrar código, nome e preço dos produtos cujo preço do produto seja maior que a média de preço de todos os produtos. **(19 linhas)**
- y. Mostrar código, nome e preço dos produtos cujo preço seja maior que a média de preço de todos os produtos e a unidade de estoque do produto seja 'kg' ou 'um' ou 'l' e tenha a letra "h" em qualquer parte do nome do produto. **(1 linha – chiclete)**

Nível 4 - Subqueries

- z. Mostrar os dados dos clientes que compraram no dia 13/01/2007 e que não compraram no dia 02/01/2007. **(3 linhas – clientes 7, 8 e 10)**
 - i. **Dica:** Se você encontrou 5 linhas, o cliente 1 apareceu no resultado, observe como está fazendo a restrição sobre a data 02/01/2007. Lembre-se das cláusulas **IN** e **NOT IN**.
- aa. Mostrar os dados dos produtos que nunca foram comprados por clientes que moram no estado do 'Rio de Janeiro'. **(30 linhas)**
- bb. Listar o nome e telefone dos clientes que já compraram 'arroz', mas que nunca compraram 'feijao'. **(4 linhas, clientes 1, 4, 7 e 8)**
- cc. Listar o nome do cliente, o telefone do cliente e a descrição do estado onde o cliente está localizado, de todos os clientes que

nunca compraram nada ou que o total comprado do cliente for menor que o total geral comprado no dia 01/01/2007, ordenando por nome de cliente ascendente. **(31 linhas)**

Nível 5 - Outer Joins

dd. Listar código, nome e preço dos produtos. Para os produtos que já foram vendidos, listar também o valor total vendido do mesmo. **(47 linhas sendo que 8 não possuem total vendido)**

i. Dica: experimente ordenar pelo total vendido em ordem ascendente ou descendente, após o nome do campo inclua **NULLS [FIRST|LAST]** para visualizar os 8 (oito) registros sem vendas.

ee. Listar o nome de cada cliente (listar todos os clientes) e o nome de cada produto que o cliente comprou ordenando o resultado por nome do cliente. **(212 linhas)**

Nível 6 - Combinação dos Níveis Anteriores (Nível Hard)

ff. Listar todos os dados do produto, mais o valor total vendido de cada produto, somente se o valor total vendido for menor que o valor total vendido de 'sal'. **(25 linhas)**

i. **Dica:** Coloque a subquerie, que calcula o valor total vendido de sal, no resultado da querie externa para poder avaliar se o resultado está correto.

gg. Selecionar o código, o nome e o valor total comprado de cada produto, somente para os produtos que tenham sido comprados por clientes que moram em 'VITORIA', e cujo valor total comprado seja maior que o somatório de valor de todos os produtos vendidos para clientes de 'VILA VELHA'. **(10 linhas)**

i. Dica: Pesquise pela cláusula **HAVING**

hh. Selecionar o código, o nome do cliente e a quantidade máxima comprada de 'arroz', somente se o cliente nunca comprou 'feijão' ou se a quantidade máxima de 'arroz' comprada pelo cliente for maior que a quantidade máxima de 'feijão' comprado pelo mesmo

cliente, ordenando o resultado por quantidade máxima comprada.

(6 linhas)

- i. **Dica 1:** Observe a forma de restrição sobre as quantidades máxima de arroz sobre as quantidades máximas de feijão.
- ii. **Dica 2:** Observe que ambas as quantidades acima (arroz e feijão) devem levar em consideração o mesmo cliente.
- ii. Crie uma VIEW contendo o nome do produto e a descrição da unidade de estoque de cada produto.
- jj. Crie uma VIEW contendo o nome de cada cliente, o valor total comprado por cada cliente para todos os clientes que morem na cidade de Vitória e cujo valor total comprado for maior que a média comprada por todos os clientes de Vila Velha.

Parte 4 – PL/SQL

Alguns SGBDs permitem o desenvolvimento de procedimentos armazenados, procedimentos esses que permitem a criação de pequenos programas, executando regras de negócios específicas que se apoiam no poder dos bancos de dados para dar suporte ao desenvolvimento de softwares. Nessa etapa iremos praticar a criação de Funções (Function), Procedimentos (Procedures) e Gatilhos (Triggers).

Seção 1 - Function

Function é um método que retorna valor

1. A sintaxe para criação de *Function* é:

Sintaxe	Exemplo
<pre>CREATE [OR REPLACE] FUNCTION function_name [(parameter [,parameter])] RETURN return_datatype IS AS [declaration_section] BEGIN executable_section [EXCEPTION exception_section] END [function_name];</pre>	<pre>CREATE OR REPLACE FUNCTION TOTAL_CLIENTE (PCODIGO_CLIENTE IN NUMERIC) RETURN NUMERIC AS VALOR_TOTAL NUMERIC; BEGIN SELECT SUM(VALOR_TOTAL) AS TOTAL_CLIENTE INTO VALOR_TOTAL FROM PEDIDOS WHERE CODIGO_CLIENTE = PCODIGO_CLIENTE; RETURN VALOR_TOTAL; EXCEPTION WHEN OTHERS THEN raise_application_error(-20001,'Erro encontrado - ' SQLCODE ' -ERROR- ' SQLERRM); END; CREATE OR REPLACE FUNCTION SOMA_MEDIA (VALOR1 IN NUMERIC, VALOR2 IN NUMERIC, MEDIA OUT NUMERIC) RETURN NUMERIC AS SOMA NUMERIC := 0; BEGIN SOMA := VALOR1 + VALOR2; MEDIA := (VALOR1 + VALOR2) / 2; RETURN SOMA; END;</pre>

Observação: ao criar *Function* ou *Procedure* você pode definir parâmetros. Há três tipos de parâmetros que podem ser declarados:

- IN** - O valor pode ser referenciado, mas não pode ser alterado.
- OUT** - O valor não pode ser referenciado, mas pode ser alterado.

- c) **IN OUT** – O valor pode ser referenciado e pode ser alterado.

2. Crie as funções conforme especificações à seguir:

- a. Crie uma função que compute a área de um retângulo;
- b. Altere a função anterior para que, além de retornar a área do retângulo, retorne por parâmetro OUT o perímetro;
- c. Crie uma função que recupere na tabela clientes o endereço concatenado (RUA, NÚMERO, BAIRRO, CIDADE, CEP e UF) para um dado cliente (CODIGO_CLIENTE);
 - i. Para essa função, é possível testá-la em uma consulta que recupere os dados do cliente. Experimente!
- d. Crie uma função que recupere o valor de um produto com um determinado acréscimo indicado pelo usuário por parâmetro;
- e. Crie uma função que:
 - i. Atualize o saldo de estoque (QUANTIDADE_ESTOQUE da tabela PRODUTOS);
 - ii. O novo saldo será alterado via quantidade informada por parâmetro;
 - iii. Caso o novo saldo seja negativo, não permita a alteração e retorne uma exceção com uma mensagem informando o saldo atual e o novo saldo;
 1. Pesquise pelo método **DBMS_OUTPUT.PUT_LINE**;
 - iv. Caso o novo saldo se mantenha maior ou igual a zero, retorne o novo saldo;

3. Para testar as funções, experimente uma das opções a seguir:

- a. Bloco PL/SQL

```
DECLARE
    result <type>;
BEGIN
    result := nome_da_funcao(parametros_se_houver);
END;
```

- b. Query dual

```
select nome_da_funcao(parametros_se_houver) from dual;
```

Seção 2 - Procedure

Procedure é um método que retorna **NÃO** valor

1. A sintaxe para criação de *Function* é:

Sintaxe	Exemplo
<pre>CREATE [OR REPLACE] PROCEDURE procedure_name [(parameter [,parameter])] IS [declaration_section] BEGIN executable_section [EXCEPTION exception_section] END [procedure_name];</pre>	<pre>CREATE OR REPLACE PROCEDURE ATUALIZA_DESCONTO_PEDIDO (PCODIGO_PEDIDO IN NUMERIC, PDESCONTO IN NUMERIC) IS VVALOR_TOTAL NUMERIC; VVALOR_LIQUIDO NUMERIC; BEGIN SELECT SUM(VALOR_TOTAL) AS TOTAL_CLIENTE INTO VVALOR_TOTAL FROM PEDIDOS WHERE CODIGO_PEDIDO = PCODIGO_PEDIDO; VVALOR_LIQUIDO := VVALOR_TOTAL - (VALOR_TOTAL * (PDESCONTO / 100)); UPDATE PEDIDOS SET VALOR_LIQUIDO = VVALOR_LIQUIDO WHERE CODIGO_PEDIDO = PCODIGO_PEDIDO; COMMIT; EXCEPTION WHEN OTHERS THEN raise_application_error(-20001,'Erro encontrado - ' SQLCODE ' -ERROR- ' SQLERRM); END;</pre>

2. Crie os procedimentos conforme especificações a seguir:

- a. Crie um procedimento que dê um aumento de preço a todos os produtos, com as seguintes características:
 - i. Faça um comando UPDATE na tabela PRODUTOS, projetando o aumento de 30% em todos os produtos;
- b. Crie um procedimento que dê um aumento de preço a todos os produtos, com as seguintes características:
 - i. Receba como parâmetro o percentual de aumento a ser dado;
 - ii. Capturar o código da unidade de estoque equivalente à descrição LITRO e armazená-la em variável local. Lembrar de declarar a variável;
 - iii. Faça um comando UPDATE na tabela PRODUTOS, projetando o aumento informado, somente para os produtos cuja unidade de estoque seja igual ao valor da variável obtida no item ii;
 - iv. Faça o controle da transação e em caso de erro, abortar a transação (ROLLBACK) enviando mensagem de erro. Caso o processamento da transação seja bem sucedido, também enviar mensagem informativa;

- c. Crie um procedimento que inclua uma linha na tabela CLIENTES:
 - i. Receba como parâmetros todos os dados obrigatórios para inserir uma linha na tabela CLIENTES;
 - ii. Faça o comando de inserção (INSERT INTO) na tabela CLIENTES a partir dos parâmetros recebidos;
 - iii. Formatar os campos alfanuméricos para maiúsculo (**Opcional**);
 - iv. Verificar se o argumento equivalente à RUA possui o prefixo "R." e substituir pela palavra RUA (**Opcional**);
 - v. Faça o controle da transação e em caso de erro, abortar a transação (ROLLBACK) enviando mensagem de erro. Caso o processamento da transação seja bem sucedido, também enviar mensagem informativa;
- d. Crie um procedimento que atualize as tabelas PEDIDOS, ITENS_PEDIDOS e PRODUTOS:
 - i. Receba como parâmetros todos os dados essenciais para que se faça a inserção de uma linha na tabela PEDIDOS e uma linha na tabela ITENS_PEDIDOS, exceto as datas.
 - ii. Faça primeiramente um comando de inserção na tabela PEDIDOS, com os seus respectivos parâmetros. Os elementos de data deverão ser obtidos de função do sistema (**SYSDATE**).
 - iii. Após a atualização da tabela PEDIDOS faça um comando de inserção na tabela ITENS_PEDIDOS, com os seus respectivos parâmetros.
 - 1. **LEMBRE-SE QUE O CODIGO_PEDIDO DA TABELA ITENS_PEDIDOS SERÁ O MESMO DA TABELA PEDIDOS;**
 - iv. Após a atualização da tabela ITENS_PEDIDOS execute um novo procedimento (descrito na letra "E" abaixo), passando como parâmetro o Código do Produto e a quantidade utilizados na inserção da tabela ITENS_PEDIDOS.
 - v. Faça controle de transação e em caso de erro, abortar a transação enviando mensagem de erro. Caso o processamento da transação seja bem sucedido, também enviar mensagem informativa.
- e. Crie um procedimento que atualize a tabela PRODUTOS:
 - i. Receba como parâmetro o código do produto a ser alterado e a quantidade
 - ii. Faça um comando UPDATE na tabela PRODUTOS, subtraindo a quantidade obtida pelo parâmetro da coluna quantidade da tabela, somente para o produto informado também como parâmetro.

- iii. Faça controle de transação e em caso de erro, abortar a transação enviando mensagem de erro. Caso o processamento da transação seja bem sucedido, também enviar mensagem informativa.
- f. Crie um procedimento que insira uma linha na tabela de CLIENTES:
 - i. Receba como parâmetros todos os dados essenciais para que se faça a inserção de uma linha na tabela CLIENTES. A coluna CODIGO_CLIENTE não deverá ser informada como argumento, pois será gerada durante a inserção do registro.
 - ii. Deverá verificar se todos os argumentos que estão direcionados para campos obrigatórios estão preenchidos. Caso algum deles esteja sem valor, formatar uma exceção e abortar (ROLLBACK) o processamento. Proponho que se formate exception distinto para cada campo testado. Além disso, mostrar uma mensagem indicando quais argumentos foram encontrados com problema. Proponho também que se avaliem todos os campos antes de efetivamente abortar o processamento.
 - iii. Faça um comando de inserção na tabela CLIENTES, com os seus respectivos parâmetros. O código do cliente será obtido pelo maior valor dessa coluna acrescida de mais um.
 - iv. Faça um comando UPDATE na tabela PRODUTOS, subtraindo a quantidade obtida pelo parâmetro da coluna quantidade da tabela, somente para o produto informado também como parâmetro.
 - v. Faça controle de transação e em caso de erro, abortar a transação enviando mensagem de erro. Caso o processamento da transação seja bem sucedido, também enviar mensagem informativa.

3. Para testar os procedimentos utilize a opção a seguir:

- a. Bloco PL/SQL

```
BEGIN
    nome_do_procedimento(parametros_se_houver);
END;
```

- b. Caso o procedimento realize alguma alteração em tabelas, é interessante que se realize uma consulta na tabela para verificar a alteração efetivada.

Seção 3 - Triggers

Trigger é um método autônomo que executa em determinado momento, podendo realizar alteração no registro antes de persisti-lo.

- Não há retorno de valor.
- Não recebe parâmetros.
- O tamanho máximo é de 32K.
- Deve-se ter o cuidado de não criar recursividade.
- No corpo pode conter comandos DML, estruturas de controle, chamadas de procedimentos/funções.
- Não é possível usar comandos DDL e controle de transação (COMMIT, ROLLBACK, SAVEPOINT).

As triggers de banco de dados são executadas implicitamente quando uma instrução INSERT, UPDATE ou DELETE (instrução de acionamento) é emitida junto da tabela, independente do usuário conectado ou da aplicação usada.

São elementos fundamentais das *triggers*: tempo, tipo e evento de acionamento.

- O **tempo** determina quando será executada, podendo ser BEFORE ou AFTER.
 - i. BEFORE: executa antes do evento de acionamento em uma tabela ser efetivado. Este tipo de trigger age analisando a alteração antes da verificação das eventuais constraints que existam na tabela.
 - ii. AFTER: executa após o evento de acionamento em uma tabela ser efetivado. Deseja-se realizar alguma ação tomando por base os dados já com a alteração consolidada e todas as restrições de integridades respeitadas.
- O **tipo** determina quantas vezes o corpo da trigger será executado. Podendo ser por instrução ou por linha:
 - i. **Instrução**: Dispara o gatilho de instrução para todo o conjunto afetado pelo evento de acionamento, mesmo que nenhuma linha seja afetada. Os gatilhos de instrução são úteis se a ação do gatilho não depender dos dados das linhas afetados ou dos dados fornecidos pelo próprio evento de acionamento.
 - ii. **Linha**: Dispara o gatilho para cada linha afetada pelo evento de acionamento. Se nenhuma linha for afetada pelo evento de acionamento, nenhum gatilho de linha será executado. Os gatilhos de linha são úteis se a ação do gatilho depender dos dados das linhas afetadas ou dos

dados fornecidos pelo próprio evento de acionamento. É determinado pela instrução **FOR EACH ROW**.

- O evento de acionamento determina a operação de manipulação de dados que dispara a trigger, podendo ser: **Insert**, **Update** e **Delete**.

1. A sintaxe para criação de *Triggers* é:

Sintaxe	Exemplo
<pre>CREATE [OR REPLACE] TRIGGER trigger_name {BEFORE AFTER} INSERT [OR UPDATE OR DELETE] ON table_name [FOR EACH ROW] DECLARE [declaration_section] BEGIN executable_section [EXCEPTION exception_section] END;</pre>	<pre>/* Objetivo do Trigger: Validar se o salário é maior que 5000 somente para pessoas cargo de Gerencia ou Presidência */ CREATE OR REPLACE TRIGGER TRIGGER_BEFORE BEFORE INSERT OR UPDATE ON empresas FOR EACH ROW BEGIN IF NOT (:NEW.CARGO IN ('GERENTE' , 'PRESIDENTE')) AND :NEW.SALARIO > 5000 THEN RAISE_APPLICATION_ERROR (-20202, 'EMPREGADO NÃO PODE GANHAR TANTO!'); END IF; END;</pre> <pre>/* Objetivo: Valida se o cadastramento do empregado está ocorrendo dentro do horário de expediente */ CREATE OR REPLACE TRIGGER TRIGGER_BEFORE1 BEFORE INSERT ON empresas BEGIN IF (TO_CHAR (sysdate,'DY') IN ('SAT','SUN')) OR (TO_CHAR(sysdate,'HH24') NOT BETWEEN '08' AND '18') THEN RAISE_APPLICATION_ERROR (-20500, 'Só é possível incluir em EMP durante expediente.');</pre> <pre>/* Objetivo: Salva os dados antes e depois de qualquer atualização em uma tabela de auditoria */ CREATE OR REPLACE TRIGGER auditoria_emp AFTER DELETE OR INSERT OR UPDATE ON empresas FOR EACH ROW BEGIN INSERT INTO audit_emp (usuario, hora_corrente, matricula, nome_antigo, nome_novo, cargo_antigo, cargo_novo, salario_antigo, salario_novo) VALUES (USER, SYSDATE, :OLD.empno, :OLD.ename, :NEW.ename, :OLD.job, :NEW.job, :OLD.sal, :NEW.sal); END;</pre>

2. Crie os gatilhos conforme especificações a seguir:

- a. Um trigger que exclua todos os itens de um pedido, quando este for excluído:
 - i. Trigger para o comando DELETE da tabela PEDIDOS
 - ii. Faça um comando de DELETE para todos os itens (ITENS_PEDIDOS) do pedido excluído
- b. Um trigger que verifique se o saldo do produto poderá se tornar negativo:
 - i. Quando um comando de alteração for emitido sobre a quantidade da tabela produto podendo torná-lo negativo.
 - ii. Caso ocorra, abortar o processo emitindo a mensagem RAISE_APPLICATION_ERROR (-20500, 'Saldo do Material poderá se tornar negativo.')
- c. Um trigger que verifique se determinada venda poderá causar saldo negativo:
 - i. Quando for feita a inserção de um novo item, validar se a quantidade vendida daquele produto é superior a quantidade disponível do produto.
 - ii. Caso ocorra, abortar o processo emitindo a mensagem RAISE_APPLICATION_ERROR (-20500, 'Saldo Indisponível')
 - iii. Acrescentar à mensagem o saldo disponível no momento;
- d. Um trigger que verifique se determinada venda poderá causar saldo negativo:
 - i. Quando for feita a inserção de um novo item, validar se a quantidade vendida daquele produto é superior a quantidade disponível do produto.
 - ii. Se for, limitar a quantidade vendida ao saldo disponível do produto.
- e. Um trigger que seja executado somente para os clientes do Espírito Santo:
 - i. Quando for inserido algum cliente para o Estado do Espírito Santo deverá ser verificado se o CEP do mesmo começa com 29.
 - ii. Caso seja diferente de 29, abortar o processo emitindo a mensagem RAISE_APPLICATION_ERROR (-20500, 'CEP inválido')
- f. Um trigger que insira dados na tabela de Histórico de Exclusões (HIST_EXC):
 - i. Trigger vinculado ao comando de DELETE da tabela CLI.

- ii. Faça um comando de INSERT na tabela HIST_EXC
- iii. Obter os dados para o comando insert das seguintes fontes:
 1. HIST_EXC_ID (será incrementado manualmente considerando o ultimo registro incluído na tabela)
 2. HIST_EXC_NOME_TAB = CLIENTES
 3. HIST_EXC_PK_EXC (Chave primária da linha que está sendo excluída na tabela).
 4. HIST_EXC_DT_EXC (Data da exclusão). Será capturada do sistema SYSDATE.
 5. HIST_EXC_LOGIN (Login da pessoa que efetuou a exclusão). Será capturado do sistema USER.

HIST_EXC			
HIST_EXC_ID	numeric(8)	<pk>	not null
HIST_EXC_NOME_TAB	char(30)		not null
HIST_EXC_PK_EXC	char(30)		not null
HIST_EXC_DT_EXC	datetime		not null
HIST_EXC_LOGIN	Char(20)		not null

```
CREATE TABLE HIST_EXC
(HIST_EXC_ID          NUMERIC(8) PRIMARY KEY,
 HIST_EXC_NOME_TAB    VARCHAR2(30) NOT NULL,
 HIST_EXC_PK_EXC      VARCHAR2(30) NOT NULL,
 HIST_EXC_DT_EXC      DATE NOT NULL,
 HIST_EXC_LOGIN       VARCHAR2(20) NOT NULL);
```

- g. Um trigger que exclua os itens que sejam do mesmo produto que está sendo excluído (usa cursor):
 - i. Trigger vinculado ao comando de DELETE a tabela PRO
 - ii. Faça um comando de DELETE no ITE correspondente ao produto excluído.
 - iii. Capture o produto excluído da SYSTEM TEST TABLE (buffer OLD)
 - iv. Se houve problema, abortar o trigger.
 - v. Senão, verificar se existe mais algum produto associado ao mesmo pedido do qual o item foi excluído.
 - vi. Se existirem outros produtos, encerrar o trigger
 - vii. Senão, ou seja o item excluído era o único do pedido, emitir um comando de exclusão da tabela PED, referente ao pedido do qual o item foi excluído.

- h. Um trigger que verifique se o valor de algum produto, após aumento, ficará maior do que a média de valor de todos os produtos.
 - i. Obter a média de valores e armazenar em uma variável para ser utilizado na item abaixo. Considerar (e tratar) que alguns preços podem estar nulos.
 - ii. Quando for feita alteração no valor do produto, verificar se o novo valor é maior do que a média de valor de todos os produtos.
 - iii. Se sim, desfazer a alteração, substituindo o novo valor pelo valor antigo.
 - iv. Gerar mensagem via `dbms_output.put_line` e não via `raise_application_error`, informando a situação.